

PHẦN 1.1: Thiết lập môi trường và Cài đặt các gói thư viện (Environment Setup)

Môi trường phát triển là tập hợp các công cụ và phần mềm cho phép chúng ta viết, kiểm tra và gỡ lỗi mã. Python là ngôn ngữ được sử dụng phổ biến nhất cho Machine Learning nhờ cú pháp đơn giản, tính linh hoạt và hệ sinh thái thư viện vô cùng mạnh mẽ.

Mặc dù các môi trường đám mây như Google Colab thường đã cài đặt sẵn phần lớn các thư viện cốt lõi cho Data Science (như `numpy`, `pandas`, `matplotlib`, `scikit-learn`), chúng ta vẫn cần cài đặt bổ sung hoặc cập nhật một số công cụ chuyên dụng để phục vụ cho các kỹ thuật Học sâu (Deep Learning) nâng cao trong chương này:

1. **TensorFlow & Keras:** Nền tảng chính để xây dựng các Mạng nơ-ron tích chập (CNN). Kể từ phiên bản 2.4, Keras đã được tích hợp chặt chẽ vào TensorFlow và trở thành API cấp cao chính thức của nó. Việc cài đặt TensorFlow sẽ tự động cài đặt Keras đi kèm.
2. **Keras Tuner:** Một thư viện cực kỳ hữu ích giúp tự động hóa quá trình tìm kiếm và tinh chỉnh các siêu tham số (hyperparameters) cho mạng nơ-ron thay vì chúng ta phải thử nghiệm thủ công.
3. **TensorBoard Plugin:** TensorBoard là một công cụ trực quan hóa tương tác tuyệt vời để theo dõi các đường cong học tập (loss/accuracy), hình dung biểu đồ tính toán và phân tích tài nguyên. Chúng ta sẽ cài thêm plugin `tensorboard-plugin-profile` để đo lường và xác định các "nút thắt cổ chai" (bottlenecks) về tốc độ của mô hình.

Lưu ý: Chúng ta sử dụng lệnh `%pip install` trực tiếp trong Jupyter Notebook để cài đặt các thư viện này vào môi trường thực thi hiện tại.

```
import sys
import platform

print(f"Phiên bản Python hiện tại: {platform.python_version()}")
print("Đang tiến hành kiểm tra và cài đặt các thư viện bổ sung...\n")

# 1. Cài đặt Keras Tuner để hỗ trợ tinh chỉnh siêu tham số mô hình Keras
# Sử dụng -q (quiet) để ẩn các log không cần thiết, -U (upgrade) để cài bản mới nhất
%pip install -q -U keras-tuner

# 2. Cài đặt plugin Profile cho TensorBoard để lập hồ sơ hiệu suất mạng nơ-ron
%pip install -q -U tensorboard-plugin-profile
```

```
# 3. Cài đặt thư viện TensorFlow Datasets để tải nhanh các bộ dữ liệu chuẩn (như C
%pip install -q -U tensorflow-datasets
```

```
# 4. (Tùy chọn) Cập nhật TensorFlow và các thư viện cơ bản nếu bạn chạy trên máy c
# (Nếu chạy trên Colab, bạn có thể bỏ qua dòng dưới đây vì hệ thống đã có sẵn bản
# %pip install -q -U tensorflow numpy pandas matplotlib scikit-learn
```

```
print("\nCài đặt hoàn tất! Môi trường đã sẵn sàng để chuyển sang bước Import thư \
```

Phiên bản Python hiện tại: 3.12.13

Đang tiến hành kiểm tra và cài đặt các thư viện bổ sung...

Cài đặt hoàn tất! Môi trường đã sẵn sàng để chuyển sang bước Import thư viện.

PHẦN 1.2: Khai báo thư viện và Thiết lập cấu hình ban đầu (Imports & Configuration)

Sau khi đã cài đặt xong các công cụ cần thiết, bước tiếp theo là khai báo (import) chúng vào không gian làm việc. Trong phần này, chúng ta sẽ thực hiện các cấu hình quan trọng sau:

- Khai báo thư viện cốt lõi:** Import `tensorflow` (thường được viết tắt là `tf`), Keras API nằm trong `tf.keras`, `numpy` để xử lý mảng tính toán, và `matplotlib.pyplot` để trực quan hóa dữ liệu và vẽ đồ thị.
- Thiết lập hạt giống ngẫu nhiên (Random Seed):** Các mạng nơ-ron khởi tạo trọng số ngẫu nhiên ban đầu. Việc thiết lập một hạt giống ngẫu nhiên cố định (ví dụ như số `42` - con số mang ý nghĩa vui vui là "Câu trả lời cho vạn vật") giúp đảm bảo kết quả huấn luyện có thể tái lập được (reproducible) giống hệt nhau trong mỗi lần bạn chạy lại notebook này. Chúng ta sẽ sử dụng hàm `tf.keras.utils.set_random_seed()` cực kỳ tiện lợi vì nó đặt seed cho cả TensorFlow, Python, và NumPy cùng lúc.
- Kiểm tra và Cấu hình phần cứng (GPU):** Việc huấn luyện các mạng CNN cho Thị giác máy tính đòi hỏi rất nhiều tài nguyên. Chạy trên GPU có thể giảm thời gian từ hàng giờ xuống còn vài phút. Đoạn mã dưới đây sẽ kiểm tra xem TensorFlow có nhận diện được GPU nào không. Ngoài ra, mặc định TensorFlow sẽ chiếm toàn bộ RAM của GPU. Chúng ta sẽ thiết lập chế độ "cấp phát bộ nhớ động" (`memory_growth`) để nó chỉ lấy thêm bộ nhớ khi thực sự cần.

```
import tensorflow as tf
import numpy as np
```

```

import matplotlib.pyplot as plt
import keras_tuner as kt
import os
import random
from pathlib import Path

# 1. Cố định hạt giống ngẫu nhiên để tái lập kết quả
seed_value = 42
tf.keras.utils.set_random_seed(seed_value) #
print(f"Đã thiết lập Random Seed: {seed_value}")

# 2. Kiểm tra phiên bản thư viện
print("Phiên bản TensorFlow:", tf.__version__)

# 3. Kiểm tra và thiết lập thiết bị GPU
physical_gpus = tf.config.list_physical_devices("GPU")
if physical_gpus:
    print(f"Tuyệt vời! Phát hiện thấy {len(physical_gpus)} GPU:")
    for gpu in physical_gpus:
        print(f" - {gpu.name}")

    # Cấu hình cấp phát bộ nhớ GPU động (Memory Growth)
    try:
        for gpu in physical_gpus:
            tf.config.experimental.set_memory_growth(gpu, True)
        print("Đã bật chế độ tự động mở rộng bộ nhớ GPU (Memory Growth).")
    except RuntimeError as e:
        # Việc thiết lập Memory Growth phải được thực hiện trước khi khởi tạo bất
        print("Lưu ý khi cấu hình GPU:", e)
else:
    print("CẢNH BÁO: Không tìm thấy GPU! Quá trình huấn luyện CNN sẽ rất chậm trên CPU.")
    print("Mẹo: Nếu bạn đang dùng Google Colab, hãy vào menu Runtime -> Change run")

Đã thiết lập Random Seed: 42
Phiên bản TensorFlow: 2.19.0
CẢNH BÁO: Không tìm thấy GPU! Quá trình huấn luyện CNN sẽ rất chậm trên CPU.
Mẹo: Nếu bạn đang dùng Google Colab, hãy vào menu Runtime -> Change runtime type ->

```

✓ PHẦN 2.1: Tải tập dữ liệu hình ảnh (Loading Dataset)

Trong bài thực hành này, chúng ta sẽ sử dụng tập dữ liệu **Fashion MNIST**. Đây là một tập dữ liệu tiêu chuẩn thường được dùng để thay thế cho tập MNIST truyền thống (nhận dạng chữ số) do có độ phức tạp cao hơn. Tập dữ liệu này bao gồm 70.000 hình ảnh thang độ xám (grayscale) có kích thước 28×28 pixel, đại diện cho các mặt hàng thời trang và được chia thành 10 lớp.

Keras cung cấp sẵn một số hàm tiện ích để lấy và tải các tập dữ liệu phổ biến một cách dễ dàng. Tập dữ liệu khi tải về đã được xáo trộn và chia sẵn thành

hai phần: một tập huấn luyện (60.000 hình ảnh) và một tập kiểm tra (10.000 hình ảnh). Tuy nhiên, trong học máy thực tế, chúng ta luôn cần một tập xác thực (validation set) để theo dõi và tránh quá khớp (overfitting) trong quá trình huấn luyện. Do đó, chúng ta sẽ giữ lại 5.000 hình ảnh cuối cùng từ tập huấn luyện để làm tập xác thực.

Khác với dữ liệu từ Scikit-Learn (thường là mảng 1D phẳng), mỗi hình ảnh tải qua Keras được giữ nguyên cấu trúc không gian là một mảng 28×28 . Hơn nữa, cường độ pixel ban đầu được biểu diễn dưới dạng số nguyên (từ 0 đến 255). Đồng thời, ta cũng cần định nghĩa danh sách tên các mặt hàng (class names) tương ứng với nhãn số từ 0 đến 9 để dễ dàng trực quan hóa sau này.

```
# Gọi hàm tiện ích của Keras để tải tập dữ liệu Fashion MNIST
fashion_mnist = tf.keras.datasets.fashion_mnist.load_data() #

# Giải nén tuple được trả về thành tập huấn luyện và tập kiểm tra ban đầu
(X_train_full, y_train_full), (X_test, y_test) = fashion_mnist #

# Trích xuất 5.000 mẫu cuối cùng từ tập huấn luyện để tạo tập xác thực (validation)
X_train, y_train = X_train_full[:-5000], y_train_full[:-5000] #
X_valid, y_valid = X_train_full[-5000:], y_train_full[-5000:] #

# Định nghĩa tên các lớp tương ứng với nhãn (từ 0 đến 9)
class_names = ["T-shirt/top", "Trouser", "Pullover", "Dress", "Coat",
               "Sandal", "Shirt", "Sneaker", "Bag", "Ankle boot"] #

# Hiển thị thông tin tổng quan về dữ liệu vừa tải
print("--- THÔNG TIN TẬP DỮ LIỆU FASHION MNIST ---")
print(f"Kích thước tập huấn luyện (Training): Khung hình {X_train.shape}, Nhãn {y_train.shape}")
print(f"Kích thước tập xác thực (Validation): Khung hình {X_valid.shape}, Nhãn {y_valid.shape}")
print(f"Kích thước tập kiểm tra (Testing): Khung hình {X_test.shape}, Nhãn {y_test.shape}")
print(f"Kiểu dữ liệu của ảnh (ban đầu): {X_train.dtype}") #
print(f"Nhãn ví dụ đầu tiên trong tập huấn luyện là '{class_names[y_train[0]]}' (nhãn số {y_train[0]})")

--- THÔNG TIN TẬP DỮ LIỆU FASHION MNIST ---
Kích thước tập huấn luyện (Training): Khung hình (55000, 28, 28), Nhãn (55000,)
Kích thước tập xác thực (Validation): Khung hình (5000, 28, 28), Nhãn (5000,)
Kích thước tập kiểm tra (Testing): Khung hình (10000, 28, 28), Nhãn (10000,)
Kiểu dữ liệu của ảnh (ban đầu): uint8
Nhãn ví dụ đầu tiên trong tập huấn luyện là 'Ankle boot' (nhãn số 9)
```

PHẦN 2.2: Chuẩn hóa và Trực quan hóa dữ liệu (Normalization & Visualization)

1. Chuẩn hóa dữ liệu (Data Normalization): Như đã thấy ở phần trước, cường độ của mỗi pixel trong ảnh ban đầu được biểu diễn bằng một số

nguyên nằm trong khoảng từ 0 (màu đen/trắng tùy hệ màu) đến 255. Tuy nhiên, các Mạng nơ-ron nhân tạo (ANN/CNN) thường hoạt động hiệu quả và hội tụ nhanh hơn nhiều khi các giá trị đầu vào được giữ ở một khoảng nhỏ, thường là quanh mốc 0. Do đó, để đơn giản hóa quá trình tiền xử lý, chúng ta sẽ chuẩn hóa dữ liệu bằng cách chia toàn bộ cường độ pixel cho 255.0, giúp chuyển đổi kiểu dữ liệu thành số thực (float) và thu hẹp dải giá trị về khoảng ``.

2. Trực quan hóa dữ liệu (Data Visualization): Việc xem xét trực tiếp một vài mẫu dữ liệu là bước không thể thiếu để đảm bảo chúng ta đã tải và gán nhãn chính xác. Chúng ta sẽ sử dụng thư viện `matplotlib` để vẽ một lưới các hình ảnh đầu tiên trong tập huấn luyện, kết hợp với danh sách `class_names` đã định nghĩa ở phần 2.1 để in tên lớp tương ứng ngay phía trên mỗi ảnh.

```
# 1. Chuẩn hóa dữ liệu (Đưa giá trị pixel từ về)
X_train, X_valid, X_test = X_train / 255.0, X_valid / 255.0, X_test / 255.0 #

# Kiểm tra lại kiểu dữ liệu và giá trị lớn nhất sau khi chuẩn hóa
print(f"Kiểu dữ liệu sau chuẩn hóa: {X_train.dtype}")
print(f"Giá trị pixel lớn nhất: {X_train.max()} và nhỏ nhất: {X_train.min()}\n")

# 2. Trực quan hóa dữ liệu
# Vẽ 10 hình ảnh đầu tiên trong tập huấn luyện
plt.figure(figsize=(12, 5))
for i in range(10):
    # Tạo một khung con (subplot) với lưới 2 hàng, 5 cột
    plt.subplot(2, 5, i + 1)

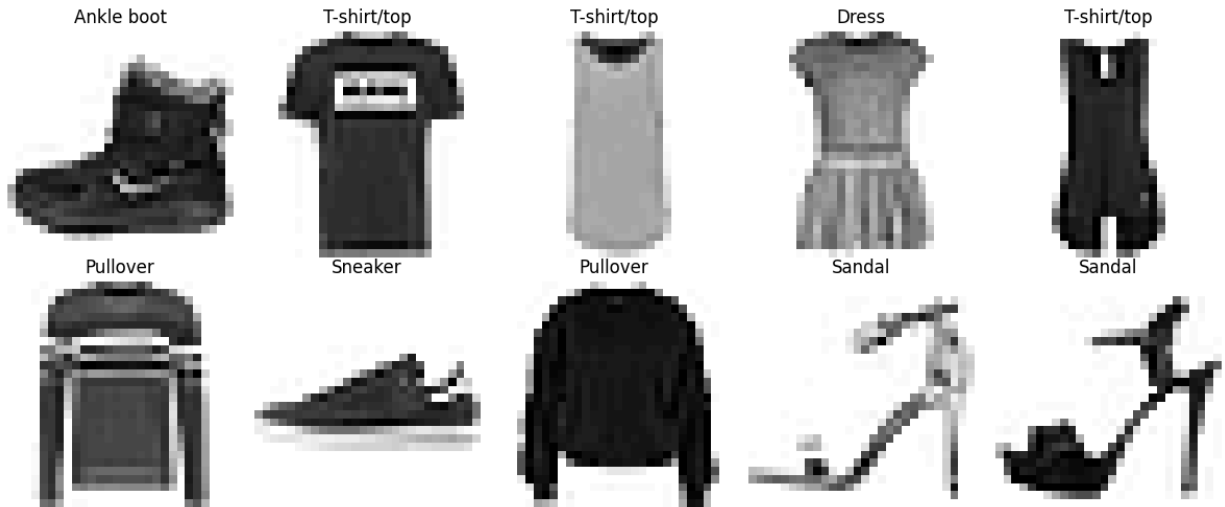
    # Hiển thị ảnh với thang độ xám (binary/grayscale)
    plt.imshow(X_train[i], cmap="binary") #

    # Lấy nhãn của ảnh và tra cứu tên lớp tương ứng
    plt.title(class_names[y_train[i]]) #

    # Ẩn trục tọa độ cho ảnh nhìn gọn gàng hơn
    plt.axis("off")

# Tự động căn chỉnh khoảng cách giữa các ảnh
plt.tight_layout()
plt.show()
```

Kiểu dữ liệu sau chuẩn hóa: float64
Giá trị pixel lớn nhất: 1.0 và nhỏ nhất: 0.0



✓ PHẦN 3.1: Mạng AlexNet - Cột mốc lịch sử & Khối trích xuất đặc trưng

1. Sự ra đời của AlexNet AlexNet được phát triển bởi Alex Krizhevsky, Ilya Sutskever và Geoffrey Hinton, đã giành chiến thắng áp đảo trong cuộc thi ILSVRC 2012 khi giảm tỷ lệ lỗi top-5 xuống chỉ còn 17%, bỏ xa đối thủ thứ hai (26%). Kiến trúc này kế thừa mạng LeNet-5 cổ điển nhưng có quy mô lớn hơn, sâu hơn và là mạng đầu tiên xếp chồng trực tiếp các lớp tích chập lên nhau thay vì bắt buộc xen kẽ một lớp gộp (pooling) sau mỗi lớp tích chập.

2. Các kỹ thuật đột phá trong phần trích xuất đặc trưng Khối trích xuất đặc trưng của AlexNet áp dụng những kỹ thuật mang tính bước ngoặt:

- **Hàm kích hoạt ReLU:** AlexNet sử dụng ReLU thay cho hàm Sigmoid hay Tanh truyền thống. Việc không bị bão hòa ở phần dương giúp tăng tốc độ hội tụ của thuật toán giảm độ dốc (gradient descent) lên rất nhiều lần.
- **Chuẩn hóa phản hồi cục bộ (Local Response Normalization - LRN):** AlexNet thêm một bước chuẩn hóa ngay sau bước ReLU của các lớp tích chập C1 và C3. Kỹ thuật này tạo ra sự kích hoạt cạnh tranh: các nơ-ron được kích hoạt mạnh nhất sẽ ức chế các nơ-ron khác nằm ở cùng vị trí trong các bản đồ đặc trưng lân cận, giúp các bản đồ đặc trưng chuyên biệt hóa và cải thiện khả năng tổng quát hóa.

Lưu ý thực hành: Tập dữ liệu gốc của AlexNet là ImageNet với ảnh màu kích thước $227 \times 227 \times 3$. Trong đoạn code dưới đây, chúng ta sẽ xây dựng phần trích xuất đặc trưng chuẩn lịch sử của AlexNet. Trong thực tế (ví dụ dùng cho

Fashion MNIST 28×28), ảnh cần được phóng to (upsample) trước khi đưa vào mạng.

```
# Khởi tạo mô hình Sequential cho AlexNet
alexnet_model = tf.keras.models.Sequential(name="AlexNet_Historical")

# --- KHỐI TRÍCH XUẤT ĐẶC TRƯNG (CONVOLUTIONAL BASE) ---

# Lớp C1: Tích chập đầu tiên với kernel lớn 11x11, stride 4 để giảm nhanh không gian
alexnet_model.add(tf.keras.layers.Conv2D(filters=96, kernel_size=(11,11), strides=4,
                                         padding='valid', activation='relu',
                                         input_shape=(227, 227, 3)))

# Bước chuẩn hóa LRN: Trong Keras, ta có thể dùng tf.nn.local_response_normalization
# Các siêu tham số gốc của AlexNet: r = 5, alpha = 0.0001, beta = 0.75, k = 2
alexnet_model.add(tf.keras.layers.Lambda(
    lambda x: tf.nn.local_response_normalization(x, depth_radius=2, alpha=0.0001, beta=0.75, k=2)
))

# Lớp S2: Gộp cực đại (Max Pooling) với kernel 3x3, stride 2
alexnet_model.add(tf.keras.layers.MaxPool2D(pool_size=(3,3), strides=2, padding='valid'))

# Lớp C3: Tích chập 5x5, padding 'same' để giữ nguyên kích thước không gian
alexnet_model.add(tf.keras.layers.Conv2D(filters=256, kernel_size=(5,5), strides=1,
                                         padding='same', activation='relu'))

# Chuẩn hóa LRN lần 2
alexnet_model.add(tf.keras.layers.Lambda(
    lambda x: tf.nn.local_response_normalization(x, depth_radius=2, alpha=0.0001, beta=0.75, k=2)
))

# Lớp S4: Gộp cực đại
alexnet_model.add(tf.keras.layers.MaxPool2D(pool_size=(3,3), strides=2, padding='valid'))

# Lớp C5, C6, C7: Các lớp tích chập 3x3 được xếp chồng trực tiếp lên nhau
alexnet_model.add(tf.keras.layers.Conv2D(filters=384, kernel_size=(3,3), strides=1,
                                         padding='valid', activation='relu'))
alexnet_model.add(tf.keras.layers.Conv2D(filters=384, kernel_size=(3,3), strides=1,
                                         padding='valid', activation='relu'))
alexnet_model.add(tf.keras.layers.Conv2D(filters=256, kernel_size=(3,3), strides=1,
                                         padding='valid', activation='relu'))

# Lớp S8: Lớp gộp cực đại cuối cùng trước khi chuyển sang mạng kết nối đầy đủ
alexnet_model.add(tf.keras.layers.MaxPool2D(pool_size=(3,3), strides=2, padding='valid'))

print("Đã xây dựng xong Khối trích xuất đặc trưng của AlexNet!")

Đã xây dựng xong Khối trích xuất đặc trưng của AlexNet!
/usr/local/lib/python3.12/dist-packages/keras/src/layers/convolutional/base_conv.py
    super().__init__(activity_regularizer=activity_regularizer, **kwargs)
```

PHẦN 3.2: Mạng AlexNet - Khối phân loại (Fully Connected) & Kỹ thuật Dropout

3. Khối Phân loại và Kỹ thuật Dropout Sau khi đi qua hàng loạt các lớp tích chập và gộp, dữ liệu không gian 2D sẽ được "làm phẳng" (Flatten) thành một vector 1D để đưa vào các lớp kết nối đầy đủ (Fully Connected - FC), đóng vai trò như một bộ phân loại truyền thống.

Kiến trúc AlexNet có 3 lớp kết nối đầy đủ ở cuối mạng. Trong đó, hai lớp đầu tiên (thường được gọi là F9 và F10, hoặc FC6 và FC7) có kích thước rất lớn với 4.096 nơ-ron mỗi lớp. Việc sử dụng số lượng tham số khổng lồ ở đây dẫn đến nguy cơ học vẹt (overfitting) rất cao.

Để giải quyết vấn đề này, các tác giả đã áp dụng một kỹ thuật mang tính đột phá thời bấy giờ là **Dropout**:

- Trong quá trình huấn luyện, tại mỗi bước, mạng sẽ ngẫu nhiên "tắt" (vô hiệu hóa) một tỷ lệ nơ-ron nhất định (ở đây là 50%) tại các lớp F9 và F10.
- Điều này buộc các nơ-ron còn lại phải tự lực trích xuất các đặc trưng hữu ích thay vì quá phụ thuộc vào một vài nơ-ron lân cận (ngăn chặn sự đồng thích nghi - co-adaptation).
- Kết quả là Dropout hoạt động như một bộ điều chuẩn (regularizer) cực kỳ mạnh mẽ, giúp mô hình tổng quát hóa tốt hơn trên dữ liệu mới.

Lưu ý: Lớp đầu ra gốc của AlexNet dùng để phân loại 1000 đối tượng của ImageNet. Tuy nhiên, để bạn có thể huấn luyện thử với tập dữ liệu Fashion MNIST đã tải ở Phần 2, chúng ta sẽ thiết lập lớp đầu ra là 10 nơ-ron tương ứng với 10 nhãn mặt hàng thời trang.

```
# --- KHỐI PHÂN LOẠI (FULLY CONNECTED CLASSIFIER) ---
```

```
# Làm phẳng (Flatten) bản đồ đặc trưng đầu ra của lớp S8 thành vector 1D
alexnet_model.add(tf.keras.layers.Flatten())
```

```
# Lớp F9 (hoặc FC6): Lớp kết nối đầy đủ với 4096 nơ-ron và hàm kích hoạt ReLU
alexnet_model.add(tf.keras.layers.Dense(units=4096, activation='relu'))
# Áp dụng kỹ thuật Dropout với tỷ lệ 50% (0.5) trong quá trình huấn luyện
alexnet_model.add(tf.keras.layers.Dropout(rate=0.5))
```

```
# Lớp F10 (hoặc FC7): Lớp kết nối đầy đủ thứ hai với 4096 nơ-ron
alexnet_model.add(tf.keras.layers.Dense(units=4096, activation='relu'))
# Tiếp tục áp dụng Dropout 50%
alexnet_model.add(tf.keras.layers.Dropout(rate=0.5))
```



```
# Lớp Đầu ra (Output Layer):
# Sử dụng 10 nơ-ron (cho Fashion MNIST) cùng hàm Softmax để xuất ra xác suất của từ
alexnet_model.add(tf.keras.layers.Dense(units=10, activation='softmax'))

# Biên dịch mô hình sơ bộ (Compile)
optimizer = tf.keras.optimizers.Adam(learning_rate=0.001)
alexnet_model.compile(loss='sparse_categorical_crossentropy',
                      optimizer=optimizer,
                      metrics=['accuracy'])

# In ra bảng tóm tắt kiến trúc của toàn bộ mạng AlexNet
print("--- TÓM TẮT KIẾN TRÚC MẠNG ALEXNET ---")
alexnet_model.summary()
```

```
--- TÓM TẮT KIẾN TRÚC MẠNG ALEXNET ---
Model: "AlexNet_Historical"
```

Layer (type)	Output Shape	Param #
conv2d (Conv2D)	(None, 55, 55, 96)	34,944
lambda (Lambda)	(None, 55, 55, 96)	0
max_pooling2d (MaxPooling2D)	(None, 27, 27, 96)	0
conv2d_1 (Conv2D)	(None, 27, 27, 256)	614,656
lambda_1 (Lambda)	(None, 27, 27, 256)	0
max_pooling2d_1 (MaxPooling2D)	(None, 13, 13, 256)	0
conv2d_2 (Conv2D)	(None, 13, 13, 384)	885,120
conv2d_3 (Conv2D)	(None, 13, 13, 384)	1,327,488
conv2d_4 (Conv2D)	(None, 13, 13, 256)	884,992
max_pooling2d_2 (MaxPooling2D)	(None, 6, 6, 256)	0
flatten (Flatten)	(None, 9216)	0
dense (Dense)	(None, 4096)	37,752,832
dropout (Dropout)	(None, 4096)	0
dense_1 (Dense)	(None, 4096)	16,781,312
dropout_1 (Dropout)	(None, 4096)	0
dense_2 (Dense)	(None, 10)	40,970

```
Total params: 58,322,314 (222.48 MB)
Trainable params: 58,322,314 (222.48 MB)
```

PHẦN 4.1: Mạng VGG16 - Sức mạnh của sự đơn giản và Cấu trúc dạng khối

1. Sự ra đời và Triết lý thiết kế của VGGNet VGGNet được phát triển bởi Karen Simonyan và Andrew Zisserman (thuộc Nhóm VGG, Đại học Oxford) và là Á quân của thử thách ILSVRC 2014. Trái ngược với các kiến trúc phức tạp, VGG nổi bật với triết lý thiết kế **cổ điển, cực kỳ đơn giản và mạch thẳng**.

Thay vì sử dụng các bộ lọc lớn như 11×11 hay 5×5 của AlexNet, VGG16 tạo ra bước ngoặt khi **đồng nhất sử dụng các bộ lọc (kernel) kích thước siêu nhỏ 3×3** trong toàn bộ mạng.

2. Tại sao lại là bộ lọc 3×3 ? Việc xếp chồng các bộ lọc nhỏ mang lại sức mạnh to lớn:

- **Tối ưu Vùng cảm nhận (Receptive Field):** Xếp chồng 2 lớp 3×3 sẽ tạo ra vùng cảm nhận tương đương với 1 lớp 5×5 ; và xếp chồng 3 lớp 3×3 sẽ tương đương với 1 lớp 7×7 .
- **Tăng tính phi tuyến:** Thay vì 1 lớp 7×7 chỉ có 1 hàm kích hoạt, việc dùng 3 lớp 3×3 cho phép chèn 3 hàm ReLU. Điều này giúp mạng học được các biểu diễn phức tạp và có tính phân biệt cao hơn.
- **Giảm tham số:** Số lượng trọng số cần huấn luyện của 3 lớp 3×3 ($3 \times 3^2 = 27$) ít hơn đáng kể so với 1 lớp 7×7 ($7^2 = 49$).

3. Cấu trúc dạng khối (Blocks) VGG16 có tổng cộng 16 lớp có trọng số (13 lớp Convolutional + 3 lớp Fully Connected). Phần trích xuất đặc trưng được chia làm **5 khối**. Kết thúc mỗi khối luôn là một lớp Max Pooling 2×2 (stride 2) để giảm một nửa không gian. Đồng thời, số lượng kênh (filters) sẽ được **nhân đôi** sau mỗi lớp gộp: $64 \rightarrow 128 \rightarrow 256 \rightarrow 512 \rightarrow 512$.

Để hiểu rõ cấu trúc của VGG16, chúng ta sẽ tự tay xây dựng 2 khối (blocks) đầu tiên sử dụng Keras Sequential API theo đúng tỷ lệ của bản gốc.

```
vgg_demo = tf.keras.models.Sequential(name="VGG16_Demo_Blocks")
```

```
# --- KHỐI 1 (Block 1) ---
```

```
# Gồm 2 lớp Tích chập  $3 \times 3$  (64 bộ lọc) và 1 lớp Gộp cực đại
```

```
# Lưu ý: padding='same' giúp duy trì kích thước ảnh sau khi tích chập
```

```
vgg_demo.add(tf.keras.layers.Conv2D(filters=64, kernel_size=(3,3), padding='same',  
                                     activation='relu', input_shape=(224, 224, 3), n  
vgg_demo.add(tf.keras.layers.Conv2D(filters=64, kernel_size=(3,3), padding='same',  
                                     activation='relu', name='block1_conv2'))
```

```
vgg_demo.add(tf.keras.layers.MaxPool2D(pool_size=(2,2), strides=(2,2), name='block1
```

```
# --- KHỐI 2 (Block 2) ---
```

```
# Gồm 2 lớp Tích chập 3x3 (số lượng bộ lọc nhân đôi lên 128) và 1 lớp Gộp cực đại
vgg_demo.add(tf.keras.layers.Conv2D(filters=128, kernel_size=(3,3), padding='same',
                                     activation='relu', name='block2_conv1'))
vgg_demo.add(tf.keras.layers.Conv2D(filters=128, kernel_size=(3,3), padding='same',
                                     activation='relu', name='block2_conv2'))
vgg_demo.add(tf.keras.layers.MaxPool2D(pool_size=(2,2), strides=(2,2), name='block2_pool'))

# In ra tóm tắt để quan sát sự thay đổi của Kích thước không gian và Số lượng kênh
print("--- CẤU TRÚC 2 KHỐI ĐẦU TIÊN CỦA VGG16 ---")
vgg_demo.summary()
```

--- CẤU TRÚC 2 KHỐI ĐẦU TIÊN CỦA VGG16 ---

Model: "VGG16_Demo_Blocks"

Layer (type)	Output Shape	Param #
block1_conv1 (Conv2D)	(None, 224, 224, 64)	1,792
block1_conv2 (Conv2D)	(None, 224, 224, 64)	36,928
block1_pool (MaxPooling2D)	(None, 112, 112, 64)	0
block2_conv1 (Conv2D)	(None, 112, 112, 128)	73,856
block2_conv2 (Conv2D)	(None, 112, 112, 128)	147,584
block2_pool (MaxPooling2D)	(None, 56, 56, 128)	0

Total params: 260,160 (1016.25 KB)

Trainable params: 260,160 (1016.25 KB)

PHẦN 4.2: Mạng VGG16 - Kế thừa tri thức với Keras Applications (Pre-trained Model)

1. Thách thức của VGG16 và Giải pháp Mặc dù VGG16 có cấu trúc đơn giản, nó lại sở hữu một khối lượng tham số khổng lồ (lên tới khoảng **138 triệu tham số**), đòi hỏi tới hàng tỷ phép toán cho mỗi lần dự đoán. Việc huấn luyện một mạng sâu như vậy từ con số 0 (from scratch) vô cùng tốn kém và cực kỳ dễ dẫn đến quá khớp (overfitting) nếu tập dữ liệu của bạn nhỏ.

Tuy nhiên, các đặc trưng do VGG16 trích xuất lại có tính ổn định và tính phân biệt cực kỳ xuất sắc. Giải pháp hoàn hảo ở đây là áp dụng **Học chuyển giao (Transfer Learning)**: Kế thừa bộ trọng số đã được huấn luyện sẵn (Pre-trained) trên tập dữ liệu khổng lồ ImageNet.

2. Triển khai VGG16 làm Mạng xương sống (Backbone) Keras cung cấp sẵn mô hình VGG16 trong module `tf.keras.applications`, cho phép bạn tải mạng này làm "bộ trích xuất đặc trưng" chỉ với một dòng code. Để làm điều này, ta cần:

- `weights='imagenet'`: Tải bộ trọng số chuẩn đã được tối ưu hóa.
- `include_top=False`: **Cắt bỏ phần đầu** (bỏ qua các lớp Fully Connected ở đỉnh mạng vốn dùng để phân loại 1.000 lớp đối tượng của ImageNet). Ta chỉ giữ lại phần thân trích xuất đặc trưng.

```
# Tải mô hình cơ sở VGG16 (Base Model) từ Keras Applications
# Lưu ý: Cần resize ảnh (ví dụ 224x224) nếu muốn sử dụng kiến trúc chuẩn
vgg16_base = tf.keras.applications.VGG16(
    weights='imagenet',          # Sử dụng trọng số đã huấn luyện trên ImageNet
    include_top=False,          # Cắt bỏ phần đầu (Fully Connected Layers)
    input_shape=(224, 224, 3)   # Kích thước đầu vào chuẩn của VGG16
)

# In ra tóm tắt kiến trúc của mạng cơ sở VGG16
print("--- TÓM TẮT MẠNG XƯƠNG SỐNG (BACKBONE) VGG16 ---")
vgg16_base.summary()

# In thêm thông tin để kiểm tra
print(f"\nTổng số lớp trong VGG16 Base (chưa bao gồm Head mới): {len(vgg16_base.layers)}
```

Downloading data from <https://storage.googleapis.com/tensorflow/keras-applications/58889256/58889256> 4s 0us/step

--- TÓM TẮT MẠNG XƯƠNG SỐNG (BACKBONE) VGG16 ---

Model: "vgg16"

Layer (type)	Output Shape	Param #
input_layer_2 (InputLayer)	(None, 224, 224, 3)	0
block1_conv1 (Conv2D)	(None, 224, 224, 64)	1,792
block1_conv2 (Conv2D)	(None, 224, 224, 64)	36,928
block1_pool (MaxPooling2D)	(None, 112, 112, 64)	0
block2_conv1 (Conv2D)	(None, 112, 112, 128)	73,856
block2_conv2 (Conv2D)	(None, 112, 112, 128)	147,584
block2_pool (MaxPooling2D)	(None, 56, 56, 128)	0
block3_conv1 (Conv2D)	(None, 56, 56, 256)	295,168
block3_conv2 (Conv2D)	(None, 56, 56, 256)	590,080
block3_conv3 (Conv2D)	(None, 56, 56, 256)	590,080
block3_pool (MaxPooling2D)	(None, 28, 28, 256)	0
block4_conv1 (Conv2D)	(None, 28, 28, 512)	1,180,160
block4_conv2 (Conv2D)	(None, 28, 28, 512)	2,359,808
block4_conv3 (Conv2D)	(None, 28, 28, 512)	2,359,808
block4_pool (MaxPooling2D)	(None, 14, 14, 512)	0
block5_conv1 (Conv2D)	(None, 14, 14, 512)	2,359,808
block5_conv2 (Conv2D)	(None, 14, 14, 512)	2,359,808
block5_conv3 (Conv2D)	(None, 14, 14, 512)	2,359,808
block5_pool (MaxPooling2D)	(None, 7, 7, 512)	0

Total params: 14,714,688 (56.13 MB)

Trainable params: 14,714,688 (56.13 MB)

Non-trainable params: 0 (0.00 B)

Tổng số lớp trong VGG16 Base (chưa bao gồm Head mới): 19

PHẦN 5.1: GoogLeNet và Đột phá mở rộng chiều ngang (Inception Module)

1. Sự ra đời của GoogLeNet GoogLeNet (hay Inception-v1) được phát triển bởi nhóm nghiên cứu tại Google và đã giành chiến thắng tại thử thách ILSVRC

2014 với tỷ lệ lỗi top-5 dưới 7%. Điều đáng kinh ngạc là dù sâu hơn rất nhiều, GoogLeNet lại sử dụng tham số cực kỳ hiệu quả: mạng chỉ có khoảng 6 triệu tham số, ít hơn 10 lần so với AlexNet (~60 triệu).

2. Mô-đun Inception: Đột phá mở rộng chiều ngang Thay vì cố gắng tìm ra kích thước bộ lọc tối ưu nhất (như VGG chỉ dùng 3×3), GoogLeNet áp dụng triết lý "dùng tất cả". Một mô-đun Inception tiêu chuẩn chia tín hiệu đầu vào thành 4 nhánh song song:

- Nhánh 1: Tích chập 1×1
- Nhánh 2: Tích chập 3×3
- Nhánh 3: Tích chập 5×5
- Nhánh 4: Gộp cực đại (Max Pooling) 3×3

Việc sử dụng đa dạng kích thước kernel giúp mạng **nắm bắt các mẫu (patterns) ở nhiều tỷ lệ khác nhau** tại cùng một cấp độ. Các bản đồ đặc trưng từ 4 nhánh sau đó được nối (concatenate) lại với nhau dọc theo chiều sâu. Tất cả các lớp này đều dùng stride = 1 và padding = "same" để đảm bảo đầu ra có cùng kích thước không gian.

3. "Phép thuật" của Tích chập 1×1 Nếu chạy trực tiếp các bộ lọc 3×3 và 5×5 , chi phí tính toán sẽ bùng nổ. Để giải quyết, GoogLeNet chèn các lớp tích chập 1×1 vào trước chúng nhằm 3 mục đích cốt lõi:

- **Lớp thắt cổ chai (Bottleneck):** Giảm số lượng bản đồ đặc trưng (giảm chiều), từ đó cắt giảm mạnh chi phí tính toán và số lượng tham số.
- **Nắm bắt đặc trưng chiều sâu:** Chụp lại các mẫu phân bố xuyên kênh (cross-channels) thay vì các đặc trưng không gian.
- **Tăng tính phi tuyến:** Hoạt động như một mạng nơ-ron hai lớp quét qua hình ảnh, tăng cường sức mạnh biểu diễn.

```
# Để thiết kế mô-đun Inception phức tạp với các nhánh song song,
# chúng ta KHÔNG THỂ dùng Sequential API, mà phải dùng Keras Functional API.

def inception_module(x, filters_1x1, filters_3x3_reduce, filters_3x3, filters_5x5_r
    """
    Hàm tạo ra một khối Inception tiêu chuẩn (có sử dụng 1x1 bottleneck).
    """
    # Nhánh 1: Chỉ dùng Tích chập 1x1
    branch1 = tf.keras.layers.Conv2D(filters_1x1, (1,1), padding='same', activation

    # Nhánh 2: Tích chập 1x1 (giảm chiều) -> Tích chập 3x3
    branch2 = tf.keras.layers.Conv2D(filters_3x3_reduce, (1,1), padding='same', act
    branch2 = tf.keras.layers.Conv2D(filters_3x3, (3,3), padding='same', activation

    # Nhánh 3: Tích chập 1x1 (giảm chiều) -> Tích chập 5x5
    branch3 = tf.keras.layers.Conv2D(filters_5x5_reduce, (1,1), padding='same', act
```

```

branch1 = tf.keras.layers.Conv2D(filters_5x5_reduce, (1,1), padding='same', activation='relu')(x)
branch3 = tf.keras.layers.Conv2D(filters_5x5, (5,5), padding='same', activation='relu')(x)

# Nhánh 4: Max Pooling 3x3 -> Tích chập 1x1 (giảm chiều)
branch4 = tf.keras.layers.MaxPooling2D((3,3), strides=(1,1), padding='same')(x)
branch4 = tf.keras.layers.Conv2D(filters_pool, (1,1), padding='same', activation='relu')(branch4)

# Nối (Concatenate) cả 4 nhánh lại với nhau dọc theo trục kênh (axis=-1)
output = tf.keras.layers.concatenate([branch1, branch2, branch3, branch4], axis=-1)

return output

# --- TEST THỬ MÔ-ĐUN INCEPTION ---
# Giả sử đầu vào là một bản đồ đặc trưng có kích thước 28x28 và sâu 192 kênh
inputs = tf.keras.layers.Input(shape=(28, 28, 192))

# Tạo một khối Inception (tham số tương đương với module Inception đầu tiên trong G)
inception_out = inception_module(inputs,
                                filters_1x1=64,
                                filters_3x3_reduce=96, filters_3x3=128,
                                filters_5x5_reduce=16, filters_5x5=32,
                                filters_pool=32)

# Tạo một Model tạm thời chỉ để in ra bảng tóm tắt
demo_inception_model = tf.keras.Model(inputs=inputs, outputs=inception_out, name="Inception Module")

print("--- TÓM TẮT MÔ-ĐUN INCEPTION ---")
demo_inception_model.summary()

```

--- TÓM TẮT MÔ-ĐUN INCEPTION ---
Model: "Inception_Module_Demo"

Layer (type)	Output Shape	Param #	Connected to
input_layer_3 (InputLayer)	(None, 28, 28, 192)	0	-
conv2d_6 (Conv2D)	(None, 28, 28, 96)	18,528	input_layer_3[0]...
conv2d_8 (Conv2D)	(None, 28, 28, 16)	3,088	input_layer_3[0]...
max_pooling2d_3 (MaxPooling2D)	(None, 28, 28, 192)	0	input_layer_3[0]...
conv2d_5 (Conv2D)	(None, 28, 28, 64)	12,352	input_layer_3[0]...
conv2d_7 (Conv2D)	(None, 28, 28, 128)	110,720	conv2d_6[0][0]
conv2d_9 (Conv2D)	(None, 28, 28, 32)	12,832	conv2d_8[0][0]
conv2d_10 (Conv2D)	(None, 28, 28, 32)	6,176	max_pooling2d_3[...
concatenate (Concatenate)	(None, 28, 28, 256)	0	conv2d_5[0][0], conv2d_7[0][0], conv2d_9[0][0], conv2d_10[0][0]

Total params: 163,696 (639.44 KB)
Trainable params: 163,696 (639.44 KB)

PHẦN 5.2: Hoàn thiện GoogLeNet - Global Average Pooling và Bộ phân loại phụ

1. Kiến trúc tổng thể của GoogLeNet Xương sống của GoogLeNet là một ngăn xếp cực sâu bao gồm **9 mô-đun Inception** được xếp chồng lên nhau, xen kẽ với các lớp gộp cực đại (Max Pooling) để giảm độ phân giải không gian của ảnh và tăng tốc độ tính toán. Mặc dù rất sâu, nhưng nhờ các lớp thắt cổ chai 1×1 , GoogLeNet chỉ có khoảng 6 triệu tham số (ít hơn 10 lần so với AlexNet).

2. Vũ khí bí mật: Global Average Pooling (GAP) Khác với AlexNet hay VGG sử dụng các lớp kết nối đầy đủ (Fully Connected/Dense) khổng lồ ở cuối mạng gây tốn hàng chục triệu tham số, GoogLeNet áp dụng một kỹ thuật mang tính bước ngoặt: **Lớp gộp trung bình toàn cục (Global Average Pooling)**. Thay

vì làm phẳng (Flatten) bản đồ đặc trưng, GAP tính giá trị trung bình của *toàn bộ* mỗi bản đồ đặc trưng. Lớp này loại bỏ hoàn toàn thông tin không gian nhưng giúp giảm triệt để số lượng tham số, hạn chế rủi ro quá khớp (overfitting) và khiến mạng cực kỳ nhẹ.

3. Bộ phân loại phụ (Auxiliary Classifiers) Đối với một mạng quá sâu, gradient khi truyền ngược (backpropagation) từ cuối mạng về đầu mạng có thể bị suy giảm và biến mất (triệt tiêu đạo hàm). Để chống lại hiện tượng này, GoogLeNet cắm thêm hai **Bộ phân loại phụ** ở phần giữa của mạng (đỉnh của mô-đun Inception thứ 3 và thứ 6). Trong quá trình huấn luyện, tổn thất (loss) từ các bộ phân loại phụ này sẽ được cộng vào tổn thất tổng thể để hỗ trợ cập nhật trọng số và đóng vai trò như một cơ chế điều chuẩn (regularization).

```
# Để minh họa, chúng ta sẽ xây dựng một phiên bản GoogLeNet thu gọn
# (sử dụng lại hàm inception_module đã định nghĩa ở Phần 5.1)

inputs = tf.keras.layers.Input(shape=(224, 224, 3))

# --- 1. PHẦN GỐC (STEM) ---
x = tf.keras.layers.Conv2D(64, (7,7), strides=2, padding='same', activation='relu')
x = tf.keras.layers.MaxPooling2D((3,3), strides=2, padding='same')(x)
x = tf.keras.layers.Conv2D(192, (3,3), strides=1, padding='same', activation='relu')
x = tf.keras.layers.MaxPooling2D((3,3), strides=2, padding='same')(x)

# --- 2. NGẮN XẾP INCEPTION (Minh họa 2 module thay vì 9 module như bản gốc) ---
# Inception 3a
x = inception_module(x, filters_1x1=64,
                    filters_3x3_reduce=96, filters_3x3=128,
                    filters_5x5_reduce=16, filters_5x5=32,
                    filters_pool=32)

# Inception 3b
x = inception_module(x, filters_1x1=128,
                    filters_3x3_reduce=128, filters_3x3=192,
                    filters_5x5_reduce=32, filters_5x5=96,
                    filters_pool=64)

# (Tùy chọn) Cắm một BỘ PHÂN LOẠI PHỤ (Auxiliary Classifier) vào giữa mạng
aux_x = tf.keras.layers.AveragePooling2D((5,5), strides=3)(x)
aux_x = tf.keras.layers.Conv2D(128, (1,1), padding='same', activation='relu')(aux_x)
aux_x = tf.keras.layers.Flatten()(aux_x)
aux_x = tf.keras.layers.Dense(1024, activation='relu')(aux_x)
aux_output = tf.keras.layers.Dense(10, activation='softmax', name='aux_output')(aux_x)

# Tiếp tục mạng chính (giảm chiều)
x = tf.keras.layers.MaxPooling2D((3,3), strides=2, padding='same')(x)

# --- 3. GLOBAL AVERAGE POOLING (Thay thế Fully Connected khổng lồ) ---
# GAP tính trung bình không gian của từng kênh, chuyển bản đồ đặc trưng (ví dụ 7x7x
x = tf.keras.layers.GlobalAveragePooling2D()(x)
```

```
x = tf.keras.layers.Dropout(0.4)(x) # Dropout 40% như bản gốc GoogLeNet

# --- 4. LỚP ĐẦU RA CHÍNH ---
main_output = tf.keras.layers.Dense(10, activation='softmax', name='main_output')(x)

# Khởi tạo mô hình với nhiều đầu ra (Multiple Outputs)
googlenet_demo = tf.keras.Model(inputs=inputs, outputs=[main_output, aux_output], name='googlenet_demo')

print("--- TÓM TẮT KIẾN TRÚC GOOGLNET (BẢN RÚT GỌN) ---")
googlenet_demo.summary()
```


Layer (type)	Output Shape	Param #	Connected to
input_layer_4 (InputLayer)	(None, 224, 224, 3)	0	-
conv2d_11 (Conv2D)	(None, 112, 112, 64)	9,472	input_layer_4[0]...
max_pooling2d_4 (MaxPooling2D)	(None, 56, 56, 64)	0	conv2d_11[0][0]
conv2d_12 (Conv2D)	(None, 56, 56, 192)	110,784	max_pooling2d_4[...
max_pooling2d_5 (MaxPooling2D)	(None, 28, 28, 192)	0	conv2d_12[0][0]
conv2d_14 (Conv2D)	(None, 28, 28, 96)	18,528	max_pooling2d_5[...
conv2d_16 (Conv2D)	(None, 28, 28, 16)	3,088	max_pooling2d_5[...
max_pooling2d_6 (MaxPooling2D)	(None, 28, 28, 192)	0	max_pooling2d_5[...
conv2d_13 (Conv2D)	(None, 28, 28, 64)	12,352	max_pooling2d_5[...
conv2d_15 (Conv2D)	(None, 28, 28, 128)	110,720	conv2d_14[0][0]
conv2d_17 (Conv2D)	(None, 28, 28, 32)	12,832	conv2d_16[0][0]

PHẦN 6.1: Đột phá chiều sâu với Mạng ResNet - Học thăng dư và Khối Residual Unit

1. Rào cản của mạng học sâu: Triệt tiêu đạo hàm Khi các mạng nơ-ron trở nên quá sâu (ví dụ hàng chục hay hàng trăm lớp), chúng gặp phải một rào cản tối ưu hóa nghiêm trọng gọi là hiện tượng Triệt tiêu đạo hàm (Vanishing Gradients) . Trong quá trình lan truyền ngược (backpropagation), tín hiệu gradient khi truyền về các lớp đầu tiên bị suy giảm đến mức gần như biến mất, khiến trọng số không được cập nhật và mạng ngừng học. Điều này khiến một mạng sâu truyền thống thậm chí còn hoạt động kém hơn một mạng nông.			
conv2d_20 (Conv2D)	(None, 28, 28, 32)	32,896	conv2d_13[0][0], conv2d_15[0][0], conv2d_17[0][0], conv2d_18[0][0]
conv2d_22 (Conv2D)	(None, 28, 28, 32)	8,224	conv2d_20[0][0]
max_pooling2d_7 (MaxPooling2D)	(None, 28, 28, 256)	0	conv2d_22[0][0]
2. Giải pháp của ResNet: Kết nối tắt (Skip Connections) và Học thăng dư Kaiyoung He và cộng sự đã giải quyết triệt để rào cản này và vô địch ILSVRC 2015 với kiến trúc ResNet (Mạng Thăng dư) có độ sâu lên tới 152 lớp với sai số top-5 chỉ 3.6%. Và khi bí mật của ResNet chính là các Kết nối tắt (Skip Connections hay Shortcut Connections) .			
conv2d_23 (Conv2D)	(None, 28, 28, 96)	76,896	conv2d_22[0][0]

Thay vì ép mạng học trực tiếp hàm mục tiêu phức tạp $h(x)$, kết nối tắt sẽ cộng thẳng tín hiệu đầu vào x vào đầu ra của một lớp cao hơn. Lúc này, mạng bị ép học hàm thặng dư (residual learning) $f(x) = h(x) - x$. Nhờ đường kết nối tắt tín hiệu gradient có một "đường cao tốc" để truyền thẳng qua hàng trăm lớp phi tuyến tính mà không bị suy giảm, giúp quá trình huấn luyện được tăng tốc đáng kể.	conv2d_24 (Conv2D) (None, 28, 28, 16, 448)	max_pooling2d_7[...]
3. Cấu tạo Đơn vị Thặng dư (Residual Unit - RU) ResNet bản chất là một ngăn xếp rất sâu của các khối thặng dư. Một khối RU cơ bản (ví dụ trong ResNet-34) gồm hai lớp tích chập 3×3 , kết hợp với Chuẩn hóa theo lô (Batch Normalization) và hàm kích hoạt ReLU không sử dụng lớp global pooling.	concatenate_2 (Concatenate) (None, 28, 28, 480)	conv2d_19[0][0], conv2d_21[0][0], conv2d_23[0][0], conv2d_24[0][0]
Tuy nhiên, có một vấn đề xảy ra: đôi khi số lượng bản đồ đặc trưng tăng gấp đôi và kích thước không gian giảm một nửa (do sử dụng stride = 2), khiến nhánh đầu vào không khớp kích thước với đầu ra. Để có thể cộng trực tiếp chúng lại, nhánh kết nối tắt sẽ được chèn thêm một lớp tích chập 1×1 có stride = 2 nhằm ép kích thước đầu vào khớp hoàn toàn với đầu ra.	average_pooling2d (AveragePooling2D) (None, 8, 8, 480)	concatenate_2[0]...
	max_pooling2d_8 (MaxPooling2D) (None, 14, 14, 480)	concatenate_2[0]...
	conv2d_25 (Conv2D) (None, 8, 8, 128)	average_pooling2d...
	global_average_pooling2d (GlobalAveragePooling2D) (None, 480)	max_pooling2d_8[...]
	dropout_2 (Dropout) (None, 480)	global_average_p...
	dense_3 (Dense) (None, 1024)	flatten_1[0][0]
	main_output (Dense) (None, 10)	dropout_2[0][0]

```
from functools import partial
```

```
# 1. Tạo một hàm Conv2D tùy chỉnh (DefaultConv2D) để giảm lặp lại code
# ResNet đồng nhất dùng bộ lọc 3x3, padding 'same', khởi tạo He và bỏ bias (do đã
DefaultConv2D = partial(tf.keras.layers.Conv2D, kernel_size=3, strides=1,
padding="same", kernel_initializer="he_normal", use_bias=f
```

```
# 2. Định nghĩa Khối Thặng dư (Residual Unit) bằng Keras Subclassing API
class ResidualUnit(tf.keras.layers.Layer):
```

```
    def __init__(self, filters, strides=1, activation="relu", **kwargs):
        super().__init__(**kwargs)
        self.activation = tf.keras.activations.get(activation)
```

```
# Đường chính (Main Path): Gồm 2 lớp Conv2D xen kẽ Batch Normalization và
self.main_layers = [
    DefaultConv2D(filters, strides=strides),
    tf.keras.layers.BatchNormalization(),
    self.activation,
    DefaultConv2D(filters),
    tf.keras.layers.BatchNormalization()
] #
```

```
# Đường tắt (Skip Connection): Xử lý sai lệch kích thước nếu strides > 1
self.skip_layers = []
if strides > 1:
    self.skip_layers = [
        DefaultConv2D(filters, kernel_size=1, strides=strides),
        tf.keras.layers.BatchNormalization()
```

```

] #

def call(self, inputs):
    Z = inputs
    # Cho dữ liệu đi qua nhánh chính
    for layer in self.main_layers:
        Z = layer(Z)

    # Cho dữ liệu đi qua nhánh tắt
    skip_Z = inputs
    for layer in self.skip_layers:
        skip_Z = layer(skip_Z)

    # Cộng gộp tín hiệu 2 nhánh (Residual Learning) và đưa qua hàm kích hoạt
    return self.activation(Z + skip_Z) #

print("Đã khởi tạo thành công lớp ResidualUnit!")

```

Đã khởi tạo thành công lớp ResidualUnit!

PHẦN 6.2: Đột phá chiều sâu với Mạng ResNet - Hoàn thiện kiến trúc ResNet-34

1. Xây dựng ngăn xếp ResNet-34 hoàn chỉnh ResNet-34 là một mạng gồm 34 lớp có trọng số, được cấu tạo bằng cách xếp chồng các Đơn vị thặng dư (Residual Units - RU) lên nhau. Mạng bắt đầu và kết thúc gần giống với kiến trúc GoogLeNet, với phần gốc là một lớp tích chập 7×7 và một lớp gộp cực đại (Max Pooling).

Ở phần giữa là một ngăn xếp rất sâu gồm các RU được chia làm 4 nhóm với số lượng bộ lọc (filters) lần lượt là: 64, 128, 256 và 512. Cụ thể, kiến trúc này chứa 3 RU xuất ra 64 bản đồ đặc trưng, 4 RU với 128 bản đồ, 6 RU với 256 bản đồ, và 3 RU với 512 bản đồ.

Ở mỗi lần chuyển nhóm, khi số lượng bộ lọc tăng gấp đôi, bước trượt (stride) sẽ được tự động chuyển thành 2 để giảm một nửa độ phân giải không gian của bản đồ đặc trưng. Ở đỉnh mạng, chúng ta tiếp tục áp dụng lớp `GlobalAvgPool1D` để nén không gian trước khi đưa vào lớp phân loại cuối cùng.

2. Ứng dụng Keras Pre-trained Model cho ResNet Mặc dù việc tự tay code kiến trúc ResNet-34 ở trên là một bài tập rất tuyệt vời để hiểu sâu về cách hoạt động của mạng, nhưng trong thực tế, chúng ta sẽ hiếm khi phải triển khai thủ công. Bạn có thể dễ dàng tải về các phiên bản ResNet mạnh mẽ hơn

(như ResNet-50, ResNet-152) đi kèm với bộ trọng số đã được huấn luyện sẵn trên ImageNet chỉ với một dòng code thông qua `tf.keras.applications`.

```
# --- 1. TỰ TAY XÂY DỰNG RESNET-34 TỪ ĐẦU ---
# (Lưu ý: Chạy đoạn code này tiếp nối với các hàm ở Phần 6.1)

resnet34_model = tf.keras.Sequential([
    # Phần gốc của mạng
    DefaultConv2D(64, kernel_size=7, strides=2, input_shape=(28, 28, 1)), #
    tf.keras.layers.BatchNormalization(), #
    tf.keras.layers.Activation("relu"), #
    tf.keras.layers.MaxPool2D(pool_size=3, strides=2, padding="same"), #
])

prev_filters = 64
# Vòng lặp ghép nối các Khối thẳng dư: 3 khối(64), 4 khối(128), 6 khối(256), 3 khối
for filters in [64] * 3 + [128] * 4 + [256] * 6 + [512] * 3: #
    # Đặt stride = 2 khi chuyển sang nhóm filter mới (số filter lớn hơn trước)
    strides = 1 if filters == prev_filters else 2 #
    resnet34_model.add(ResidualUnit(filters, strides=strides)) #
    prev_filters = filters #

# Phần đỉnh mạng: Global Average Pooling và Đầu ra phân loại
resnet34_model.add(tf.keras.layers.GlobalAvgPool2D()) #
resnet34_model.add(tf.keras.layers.Flatten()) #
# Dùng 10 units với kích hoạt softmax để giải bài toán phân loại (như Fashion MNIST)
resnet34_model.add(tf.keras.layers.Dense(10, activation="softmax")) #

print("--- TÓM TẮT KIẾN TRÚC RESNET-34 TỰ XÂY DỰNG ---")
resnet34_model.summary()

# -----
# --- 2. SỬ DỤNG MÔ HÌNH PRE-TRAINED RESNET-50 CỦA KERAS ---

print("\nĐang tải mô hình ResNet-50 Pre-trained từ Keras Applications...")
# Tải ResNet-50 đã huấn luyện trên ImageNet
pretrained_resnet = tf.keras.applications.ResNet50(weights="imagenet") #

# Nếu muốn tận dụng ResNet-50 làm Mạng xương sống (Backbone) cho Học chuyển giao,
# hãy thêm tham số include_top=False như đã học ở Phần 4.2
resnet_backbone = tf.keras.applications.ResNet50(weights="imagenet", include_top=False)

print("\nTải thành công! Kiến trúc ResNet hiện đại đã sẵn sàng cho Transfer Learning")
```



```

/usr/local/lib/python3.12/dist-packages/keras/src/layers/convolutional/base_conv.
    super().__init__(activity_regularizer=activity_regularizer, **kwargs)
--- TÓM TẮT KIẾN TRÚC RESNET-34 TỰ XÂY DỰNG ---
Model: "sequential"

```

Layer (type)	Output Shape	Param #
conv2d_26 (Conv2D)	(None, 14, 14, 64)	3,136
batch_normalization_10 (BatchNormalization)	(None, 14, 14, 64)	256
1. Sự dịch chuyển sang Edge Computing (Điện toán biên)	(None, 14, 14, 64)	0
max_pooling2d_9 (MaxPooling2D)	(None, 7, 7, 64)	0
residual_unit_1 (ResidualUnit)	(None, 7, 7, 64)	74,240
residual_unit_1 (ResidualUnit)	(None, 7, 7, 64)	74,240
residual_unit_2 (ResidualUnit)	(None, 7, 7, 64)	74,240
MobileNet đã được phát triển Mục tiêu của MobileNet là tạo ra một kiến trúc tinh gọn, đạt được sự cân bằng tối ưu giữa kích thước mô hình nhỏ (Low Memory Footprint) và tốc độ suy luận nhanh (Low Latency). Triết lý cốt lõi ở đây là nguyên lý đánh đổi (trade-off): chấp nhận hy sinh một phần rất nhỏ độ chính xác để giảm thiểu theo cấp số nhân khối lượng tham số và phép toán.	(None, 4, 4, 128)	230,912
residual_unit_4 (ResidualUnit)	(None, 4, 4, 128)	295,936
residual_unit_5 (ResidualUnit)	(None, 4, 4, 128)	295,936
residual_unit_6 (ResidualUnit)	(None, 4, 4, 128)	295,936
residual_unit_7 (ResidualUnit)	(None, 2, 2, 256)	920,576
2. Vũ khí bí mật: Tích chập tách biệt chiều sâu (Depthwise Separable Convolution)	(None, 2, 2, 256)	1,181,696
residual_unit_9 (ResidualUnit)	(None, 2, 2, 256)	1,181,696
residual_unit_10 (ResidualUnit)	(None, 2, 2, 256)	1,181,696
residual_unit_11 (ResidualUnit)	(None, 2, 2, 256)	1,181,696
residual_unit_12 (ResidualUnit)	(None, 1, 1, 512)	3,676,160
residual_unit_14 (ResidualUnit)	(None, 1, 1, 512)	4,722,688
residual_unit_15 (ResidualUnit)	(None, 1, 1, 512)	4,722,688
global_average_pooling2d_1 (GlobalAveragePooling2D)	(None, 512)	0
flatten_2 (Flatten)	(None, 512)	0
dense_4 (Dense)	(None, 10)	5,130
total_params: 21,283,530 (81.19 MB)		
non_trainable_params: 17,024 (66.50 KB)		

Để chứng minh sức mạnh của Tích chập tách biệt chiều sâu,

```
# chúng ta sẽ so sánh trực tiếp số lượng tham số của nó với Tích chập tiêu chuẩn.

# Giả sử chúng ta có một ảnh (hoặc bản đồ đặc trưng) đầu vào với 64 kênh
input_shape = (128, 128, 64)
inputs = tf.keras.layers.Input(shape=input_shape)

# 1. TÍCH CHẬP TIÊU CHUẨN (Standard Convolution)
# Dùng bộ lọc 3x3, xuất ra 128 kênh
standard_conv = tf.keras.layers.Conv2D(filters=128, kernel_size=(3,3), padding='same')
model_standard = tf.keras.Model(inputs, standard_conv, name="Standard_Conv")

# 2. TÍCH CHẬP TÁCH BIỆT CHIỀU SÂU (Depthwise Separable Convolution)
# Keras cung cấp sẵn lớp SeparableConv2D kết hợp cả 2 bước (Depthwise + Pointwise)
separable_conv = tf.keras.layers.SeparableConv2D(filters=128, kernel_size=(3,3), padding='same')
model_separable = tf.keras.Model(inputs, separable_conv, name="Separable_Conv")

# --- SO SÁNH SỐ LƯỢNG THAM SỐ ---
print("--- SO SÁNH SỐ LƯỢNG THAM SỐ ---")
print(f"1. Tích chập Tiêu chuẩn:      {model_standard.count_params():,} tham số")
print(f"2. Tích chập Tách biệt:      {model_separable.count_params():,} tham số")

# Tính toán tỷ lệ giảm
reduction_ratio = model_standard.count_params() / model_separable.count_params()
print(f"=> Tích chập tách biệt giúp giảm ~{reduction_ratio:.1f} lần số lượng tham số")

# Nếu muốn tự xây dựng chi tiết từng bước của Tích chập tách biệt chiều sâu:
# Bước 1: Depthwise
x = tf.keras.layers.DepthwiseConv2D(kernel_size=(3,3), padding='same')(inputs)
# Bước 2: Pointwise (Conv2D 1x1)
custom_separable_out = tf.keras.layers.Conv2D(filters=128, kernel_size=(1,1))(x)

model_custom_separable = tf.keras.Model(inputs, custom_separable_out)
print(f"Số tham số nếu tự chia 2 bước: {model_custom_separable.count_params():,} tham số")

--- SO SÁNH SỐ LƯỢNG THAM SỐ ---
1. Tích chập Tiêu chuẩn:      73,856 tham số
2. Tích chập Tách biệt:      8,896 tham số
=> Tích chập tách biệt giúp giảm ~8.3 lần số lượng tham số!

Số tham số nếu tự chia 2 bước: 8,960 tham số (Khớp với SeparableConv2D)
```

✓ PHẦN 7.2: Sự tiến hóa lên MobileNetV2 và Ma trận lựa chọn mô hình

1. Cải tiến của MobileNetV2: Khối thắt cổ chai đảo ngược (Inverted Residuals) Nếu như ResNet truyền thống thiết kế một khối thặng dư theo trình tự: *Thu hẹp (bằng tích chập 1x1) -> Tích chập (3x3) -> Mở rộng (bằng 1x1)*, thì MobileNetV2 thiết kế lại hoàn toàn luồng dữ liệu này theo hướng ngược lại (đảo ngược) nhằm bảo toàn tối đa thông tin. Một khối MobileNetV2 bao gồm 3 bước:

- **Bước 1 - Mở rộng (Expand):** Dùng tích chập 1×1 để tăng mạnh số lượng kênh (ngược với ResNet).
- **Bước 2 - Trích xuất (Depthwise):** Dùng tích chập chiều sâu 3×3 trên không gian kênh đã được mở rộng.
- **Bước 3 - Thu hẹp (Squeeze / Linear Bottleneck):** Dùng tích chập 1×1 để nén kênh lại. Lớp này đặc biệt **không sử dụng hàm kích hoạt phi tuyến** (như ReLU) vì việc áp dụng phi tuyến ép dữ liệu vào một không gian chiều thấp sẽ làm phá hủy thông tin.

Cuối cùng, một kết nối tắt (Skip Connection) được cộng vào giữa đầu vào và đầu ra của khối nếu stride bằng 1, giúp gradient truyền mượt mà.

2. Ma trận Quyết định: Lựa chọn mô hình kiến trúc phù hợp Không có mạng CNN nào là hoàn hảo tuyệt đối. Việc lựa chọn phụ thuộc vào ưu tiên và ràng buộc phần cứng của dự án:

- **Bảng thông/Bộ nhớ hạn hẹp (Mobile/IoT):** Nên chọn **MobileNet** hoặc **SqueezeNet** vì chúng tối ưu hóa số lượng tham số và phép toán bậc nhất.
- **Tối đa hóa độ chính xác:** Nên chọn **EfficientNet** (áp dụng quy luật mở rộng kép - Compound Scaling) hoặc các mạng ResNeXt lớn.
- **Tốc độ thực thi trên CPU:** Khuyến dùng **Inception-v3** vì các nhánh tính toán song song của mô-đun Inception tối ưu rất tốt trên luồng của CPU.
- **Tốc độ thực thi trên GPU:** Khuyến dùng **Họ mạng ResNet** vì luồng dữ liệu thẳng, đồng nhất giúp tính toán mảng song song trên GPU cực kỳ nhanh.

Chúng ta sẽ tự tay triển khai một Khối thắt cổ chai đảo ngược (Inverted Residual # cốt lõi của mạng MobileNetV2 bằng Keras Functional API.

```
def inverted_residual_block(x, expand_filters, squeeze_filters, stride=1):
    inputs = x

    # Bước 1: Mở rộng (Expand) sử dụng Conv2D 1x1
    x = tf.keras.layers.Conv2D(expand_filters, kernel_size=(1,1), padding='same', u
    x = tf.keras.layers.BatchNormalization()(x)
    # MobileNetV2 thường sử dụng ReLU6 (giới hạn giá trị ở mức 6) để tăng độ ổn định
    x = tf.keras.layers.ReLU(max_value=6.0)(x)

    # Bước 2: Trích xuất (Depthwise) sử dụng DepthwiseConv2D 3x3
    x = tf.keras.layers.DepthwiseConv2D(kernel_size=(3,3), strides=stride, padding=
    x = tf.keras.layers.BatchNormalization()(x)
    x = tf.keras.layers.ReLU(max_value=6.0)(x)

    # Bước 3: Thu hẹp (Squeeze - Linear Bottleneck) sử dụng Conv2D 1x1
```

```

# --- TEST THỬ KHỐI MOBILENETV2 ---
# Giả sử đầu vào là bản đồ đặc trưng kích thước 56x56 với 32 kênh
inputs = tf.keras.layers.Input(shape=(56, 56, 32))

# Tạo khối: Mở rộng từ 32 lên 192 kênh, sau đó thu hẹp về 32 kênh
outputs = inverted_residual_block(inputs, expand_filters=192, squeeze_filters=32, s
demo_mobilenet_block = tf.keras.Model(inputs, outputs, name="Inverted_Residual_Bloc

print("--- TÓM TẮT KHỐI THẮT CỔ CHAI ĐẢO NGƯỢC (MOBILENET V2) ---")
demo_mobilenet_block.summary()

```

--- TÓM TẮT KHỐI THẮT CỔ CHAI ĐẢO NGƯỢC (MOBILENET V2) ---

Model: "Inverted_Residual_Block_MobileNetV2"

Layer (type)	Output Shape	Param #	Connected to
input_layer_9 (InputLayer)	(None, 56, 56, 32)	0	-
conv2d_64 (Conv2D)	(None, 56, 56, 192)	6,144	input_layer_9[0]...
batch_normalizatio... (BatchNormalizatio...	(None, 56, 56, 192)	768	conv2d_64[0][0]
re_lu (ReLU)	(None, 56, 56, 192)	0	batch_normalizat...
depthwise_conv2d_1 (DepthwiseConv2D)	(None, 56, 56, 192)	1,728	re_lu[0][0]
batch_normalizatio... (BatchNormalizatio...	(None, 56, 56, 192)	768	depthwise_conv2d...
re_lu_1 (ReLU)	(None, 56, 56, 192)	0	batch_normalizat...
conv2d_65 (Conv2D)	(None, 56, 56, 32)	6,144	re_lu_1[0][0]
batch_normalizatio... (BatchNormalizatio...	(None, 56, 56, 32)	128	conv2d_65[0][0]
add (Add)	(None, 56, 56, 32)	0	input_layer_9[0]... batch_normalizat...

Total params: 15,680 (61.25 KB)
Trainable params: 14,848 (58.00 KB)
Non-trainable params: 832 (3.25 KB)

PHẦN 8.1: Cơ chế chú ý (Attention) và Mạng nén - kích thích SENet (Squeeze-and-Excitation)

1. Sự ra đời của SENet và Cơ chế Attention Mạng nén và kích thích (Squeeze-and-Excitation Network - SENet) là kiến trúc đã giành chiến thắng vang dội tại thử thách ILSVRC 2017 với tỷ lệ lỗi top-5 giảm xuống mức đáng kinh ngạc chỉ còn **2.25%**.

Điểm đặc biệt của SENet là nó không phải là một mạng độc lập hoàn toàn mới, mà là một mô-đun (vi mạng) vô cùng linh hoạt có thể được "cắm thêm" vào bất kỳ kiến trúc hiện có nào như Inception hay ResNet (tạo thành SE-Inception và SE-ResNet). SENet áp dụng **Cơ chế chú ý (Attention)**: thay vì tìm kiếm các mẫu không gian, nó bỏ qua không gian và chỉ tập trung phân tích dọc theo chiều sâu (depth) của bản đồ đặc trưng. Nó tự học xem các đặc trưng nào thường xuất hiện cùng nhau để từ đó đánh giá lại mức độ quan trọng của từng kênh.

2. Giải phẫu cấu trúc Khối SE (SE Block) Một Khối SE hoạt động như một vi mạng nội bộ gồm 3 bước cơ bản:

- **Bước 1 - Nén (Squeeze):** Sử dụng lớp **Gộp trung bình toàn cục (Global Average Pooling)** để nén thông tin không gian của mỗi bản đồ đặc trưng thành một số duy nhất. Ví dụ: 256 bản đồ đặc trưng sẽ bị nén thành một vector 256 chiều, đại diện cho phân bố đặc trưng tổng quát.
- **Bước 2 - Kích thích (Excitation):** Vector này đi qua một lớp ẩn (Dense) kết hợp hàm kích hoạt **ReLU**. Số lượng nơ-ron ở lớp này được giảm mạnh (thường chia cho tỷ lệ $r = 16$) để tạo ra một nút thắt cổ chai, buộc mạng phải học sự kết hợp cốt lõi giữa các kênh.
- **Bước 3 - Hiệu chỉnh lại (Recalibration):** Dữ liệu đi qua một lớp đầu ra (Dense) với hàm kích hoạt **Sigmoid** để khôi phục lại số kênh ban đầu, xuất ra một "vector hiệu chỉnh" với mỗi giá trị nằm trong khoảng từ 0 đến 1. Cuối cùng, các bản đồ đặc trưng gốc được nhân với vector này. Đặc trưng quan trọng (điểm gần 1) sẽ được **tăng cường**, trong khi đặc trưng không liên quan sẽ bị **bóp nghẹt**.

Tự tay triển khai Khối Squeeze-and-Excitation (SE Block) bằng Keras Functional API

```
def se_block(inputs, ratio=16):  
    """  
    Tạo một khối Squeeze-and-Excitation (SE).  
    - inputs: Tensor đầu vào có kích thước (batch_size, height, width, channels)  
    - ratio: Tỷ lệ nén ở lớp ẩn (mặc định là 16)
```

```

"""
# Lấy số lượng kênh (channels) từ tensor đầu vào
channels = inputs.shape[-1]

# BƯỚC 1: Squeeze (Nén)
# Chuyển đổi (height, width, channels) -> (channels)
se = tf.keras.layers.GlobalAveragePooling2D(name="se_squeeze")(inputs)

# BƯỚC 2: Excitation (Kích thích)
# Thắt cổ chai với tỷ lệ ratio (vd: 256 kênh -> 16 kênh) và hàm kích hoạt ReLU
se = tf.keras.layers.Dense(channels // ratio, activation='relu', name="se_excit

# Khôi phục số kênh (vd: 16 kênh -> 256 kênh) và dùng Sigmoid để xuất vector hi
se = tf.keras.layers.Dense(channels, activation='sigmoid', name="se_excitation_

# BƯỚC 3: Recalibration (Hiệu chỉnh lại)
# Reshape lại vector (channels) thành (1, 1, channels) để có thể nhân broadcast
se = tf.keras.layers.Reshape((1, 1, channels), name="se_reshape")(se)

# Nhân bản đồ đặc trưng ban đầu với vector hiệu chỉnh
outputs = tf.keras.layers.Multiply(name="se_recalibration")([inputs, se])

return outputs

# --- TEST THỬ KHỐI SE ---
# Giả sử chúng ta có một bản đồ đặc trưng đầu vào kích thước 56x56 với 256 kênh
inputs = tf.keras.layers.Input(shape=(56, 56, 256))
se_outputs = se_block(inputs, ratio=16)

demo_se_model = tf.keras.Model(inputs, se_outputs, name="SE_Block_Demo")

print("--- TÓM TẮT KIẾN TRÚC KHỐI SQUEEZE-AND-EXCITATION ---")
demo_se_model.summary()

```

--- TÓM TẮT KIẾN TRÚC KHỐI SQUEEZE-AND-EXCITATION ---

Model: "SE_Block_Demo"

Layer (type)	Output Shape	Param #	Connected to
input_layer_10 (InputLayer)	(None, 56, 56, 256)	0	-
se_squeeze (GlobalAveragePool...	(None, 256)	0	input_layer_10[0...
se_excitation_1 (Dense)	(None, 16)	4,112	se_squeeze[0][0]
se_excitation_2 (Dense)	(None, 256)	4,352	se_excitation_1[...
se_reshape (Reshape)	(None, 1, 1, 256)	0	se_excitation_2[...
se_recalibration (Multiply)	(None, 56, 56, 256)	0	input_layer_10[0... se_reshape[0][0]

Total params: 8,464 (33.06 KB)

Trainable params: 8,464 (33.06 KB)

Non-trainable params: 0 (0.00 B)

PHẦN 8.2: Tính tương thích linh hoạt - Xây dựng SE-ResNet và SE-Inception

1. Triết lý "Cắm và Chạy" (Plug-and-Play) Như đã tìm hiểu ở phần trước, mạng SENet đã giành chiến thắng tại ILSVRC 2017 với tỷ lệ lỗi top-5 ở mức đáng kinh ngạc là 2.25%. Điều làm nên sự vĩ đại của SENet không phải là việc đập đi xây lại từ đầu, mà là việc nó mở rộng và tăng cường các kiến trúc xuất sắc hiện có (như mạng Inception và ResNet).

Bằng cách đính kèm một Khối SE nhỏ gọn vào mọi mô-đun Inception hoặc Đơn vị Thặng dư (Residual Unit) trong kiến trúc gốc, chúng ta tạo ra các phiên bản mạng mới mạnh mẽ hơn được gọi tương ứng là **SE-Inception** và **SE-ResNet**.

2. Tích hợp Khối SE vào Đơn vị Thặng dư (SE-ResNet Unit) Khối SE hoạt động như một cơ chế "tự học" xem các đặc trưng nào thường xuất hiện cùng nhau. Đối với mạng ResNet, sự tích hợp diễn ra như sau:

- Tín hiệu đi qua nhánh chính gồm các lớp tích chập (Conv2D) và chuẩn hóa (BatchNorm) như bình thường.
- Thay vì cộng trực tiếp nhánh chính này với kết nối tắt (Skip Connection), đầu ra của nhánh chính sẽ được đưa qua Khối SE.

- Khối SE thực hiện "nén và kích thích", xuất ra các bản đồ đặc trưng đã được hiệu chỉnh lại (Recalibrated feature maps).
- Cuối cùng, các bản đồ đặc trưng đã hiệu chỉnh này mới được cộng gộp với kết nối tắt và đi qua hàm kích hoạt ReLU cuối cùng.

```
# Để minh họa sự linh hoạt, chúng ta sẽ viết một hàm kết hợp một Đơn vị Thặng dư (R
# với Khối SE (se_block đã định nghĩa ở Phần 8.1) để tạo thành một Đơn vị SE-ResNet

def se_residual_unit(inputs, filters, strides=1, ratio=16):
    """
    Tạo một Đơn vị Thặng dư có tích hợp cơ chế Squeeze-and-Excitation (SE-ResNet Un
    """
    # --- NHÁNH CHÍNH (MAIN PATH) ---
    x = tf.keras.layers.Conv2D(filters, (3,3), strides=strides, padding='same', use
    x = tf.keras.layers.BatchNormalization()(x)
    x = tf.keras.layers.Activation('relu')(x)

    x = tf.keras.layers.Conv2D(filters, (3,3), strides=1, padding='same', use_bias=
    x = tf.keras.layers.BatchNormalization()(x)

    # --- TÍCH HỢP KHỐI SE ---
    # Thay vì cộng ngay với Skip Connection, ta cho x đi qua khối SE để "hiệu chỉnh
    x = se_block(x, ratio=ratio) # Gọi lại hàm se_block từ Phần 8.1

    # --- KẾT NỐI TẮT (SKIP CONNECTION) ---
    skip = inputs
    if strides > 1 or inputs.shape[-1] != filters:
        skip = tf.keras.layers.Conv2D(filters, (1,1), strides=strides, padding='sam
        skip = tf.keras.layers.BatchNormalization()(skip)

    # Cộng gộp tín hiệu đã hiệu chỉnh với nhánh tắt
    outputs = tf.keras.layers.Add()([x, skip])
    outputs = tf.keras.layers.Activation('relu')(outputs)

    return outputs

# --- TEST THỬ ĐƠN VỊ SE-RESNET ---
# Giả sử đầu vào là bản đồ đặc trưng 56x56 với 64 kênh, ta muốn xuất ra 128 kênh (k
se_resnet_inputs = tf.keras.layers.Input(shape=(56, 56, 64))
se_resnet_outputs = se_residual_unit(se_resnet_inputs, filters=128, strides=2, rati

demo_se_resnet_model = tf.keras.Model(se_resnet_inputs, se_resnet_outputs, name="SE

print("---- TÓM TẮT KIẾN TRÚC ĐƠN VỊ SE-RESNET ----")
demo_se_resnet_model.summary()
```


--- TÓM TẮT KIẾN TRÚC ĐƠN VỊ SE-RESNET ---

Model: "SE_ResNet_Unit_Demo"

Layer (type)	Output Shape	Param #	Connected to
input_layer_11 (InputLayer)	(None, 56, 56, 64)	0	-
conv2d_66 (Conv2D)	(None, 28, 28, 128)	73,728	input_layer_11[0]...
batch_normalizatio... (BatchNormalizatio...	(None, 28, 28, 128)	512	conv2d_66[0][0]
activation_1 (Activation)	(None, 28, 28, 128)	0	batch_normalizat...
conv2d_67 (Conv2D)	(None, 28, 28, 128)	147,456	activation_1[0][...
batch_normalizatio... (BatchNormalizatio...	(None, 28, 28, 128)	512	conv2d_67[0][0]
se_squeeze (GlobalAveragePool...	(None, 128)	0	batch_normalizat...
se_excitation_1 (Dense)	(None, 8)	1,032	se_squeeze[0][0]
se_excitation_2 (Dense)	(None, 128)	1,152	se_excitation_1[...
se_reshape (Reshape)	(None, 1, 1, 128)	0	se_excitation_2[...
conv2d_68 (Conv2D)	(None, 28, 28, 128)	8,192	input_layer_11[0]...
se_recalibration (Multiply)	(None, 28, 28, 128)	0	batch_normalizat... se_reshape[0][0]
batch_normalizatio... (BatchNormalizatio...	(None, 28, 28, 128)	512	conv2d_68[0][0]
add_1 (Add)	(None, 28, 28, 128)	0	se_recalibration... batch_normalizat...
activation_2 (Activation)	(None, 28, 28, 128)	0	add_1[0][0]

Total params: 233,096 (910.53 KB)

Trainable params: 232,328 (907.53 KB)

Non-trainable params: 768 (3.00 KB)

PHẦN 9.1: Học chuyển giao (Transfer Learning) - Triết lý "Đứng trên vai người khổng lồ" và Trích xuất đặc trưng

1. Triết lý Học chuyển giao (Transfer Learning) Thay vì huấn luyện một mô hình từ con số 0 (từ đầu) vốn cực kỳ tốn kém và dễ dẫn đến quá khớp (overfit) khi dữ liệu nhỏ, Học chuyển giao áp dụng triết lý "Đứng trên vai người khổng lồ". Kỹ thuật này tận dụng một mô hình đã được huấn luyện sẵn (Pre-trained CNN) trên các tập dữ liệu khổng lồ (như ImageNet) để giải quyết một bài toán mới tương tự.

Nguyên lý đằng sau là **Phân cấp đặc trưng**: Các lớp thấp của mạng CNN học được các cấu trúc cơ bản phổ quát cho mọi hình ảnh (như góc, cạnh, màu sắc), trong khi các lớp giữa và cao học các đặc trưng phức tạp hơn. Do đó, ta có thể tái sử dụng phần lớn các lớp trích xuất đặc trưng này và chỉ cần thay thế lớp đầu ra (Head) để phù hợp với bài toán mới.

2. Giai đoạn 1: Đóng băng trọng số (Freezing Layers / Feature Extraction)

Để triển khai Transfer Learning, quy trình cơ bản gồm các bước:

- **Tải Base Model & Cắt bỏ phần đầu:** Gọi một mô hình (ví dụ: Xception hoặc VGG16) với cấu hình tải trọng số chuẩn (`weights='imagenet'`) và loại bỏ các lớp Fully Connected ở đỉnh mạng bằng lệnh `include_top=False`.
- **Thiết kế Bộ phân loại mới (New Head):** Thêm lớp `GlobalAveragePooling2D` để chuyển bản đồ đặc trưng thành mảng 1D, sau đó nối với một lớp `Dense` có hàm Softmax (hoặc Sigmoid) tương ứng với số lớp của bài toán mới.
- **Đóng băng trọng số:** Khóa toàn bộ trọng số của mô hình cơ sở (`layer.trainable = False`). Mục đích cốt lõi là giữ nguyên vẹn tri thức đã học, giúp thuật toán hạ gradient chỉ tập trung cập nhật các trọng số mới được khởi tạo ngẫu nhiên ở lớp phân loại trên cùng, đảm bảo hội tụ nhanh và tránh phá hủy các đặc trưng tinh tế đã được mượn.

```
import tensorflow as tf

# Giả sử bài toán mới của chúng ta là phân loại 5 loại hoa
n_classes = 5

# 1. Tải mô hình cơ sở (Xception) đã được huấn luyện trên ImageNet
# Lệnh include_top=False bỏ qua các lớp Fully Connected ở đỉnh mạng, chỉ giữ lại ph
base_model = tf.keras.applications.xception.Xception(weights="imagenet", include_to

# 2. Đóng băng toàn bộ trọng số của mô hình cơ sở (Feature Extraction)
for layer in base_model.layers:
    layer.trainable = False #

# 3. Thiết kế Đầu phân loại mới (New Head)
# Sử dụng GlobalAveragePooling2D để chuyển bản đồ đặc trưng (Feature maps) thành ve
```

```
avg = tf.keras.layers.GlobalAveragePooling2D()(base_model.output) #
# Lớp phân loại cuối cùng cho bài toán mới
output = tf.keras.layers.Dense(n_classes, activation="softmax")(avg) #

# Lắp ráp thành mô hình hoàn chỉnh
model_transfer = tf.keras.Model(inputs=base_model.input, outputs=output) #

# 4. Biên dịch mô hình (Compile)
# Lúc này chỉ có trọng số của lớp Đầu ra (Dense) mới được cập nhật trong quá trình
optimizer = tf.keras.optimizers.SGD(learning_rate=0.1, momentum=0.9)
model_transfer.compile(loss="sparse_categorical_crossentropy",
                      optimizer=optimizer,
                      metrics=["accuracy"]) #

print("--- TÓM TẮT MÔ HÌNH HỌC CHUYỂN GIAO (GIAI ĐOẠN ĐÓNG BĂNG) ---")
# Bạn sẽ thấy phần lớn tham số được đánh dấu là "Non-trainable params"
model_transfer.summary()
```

